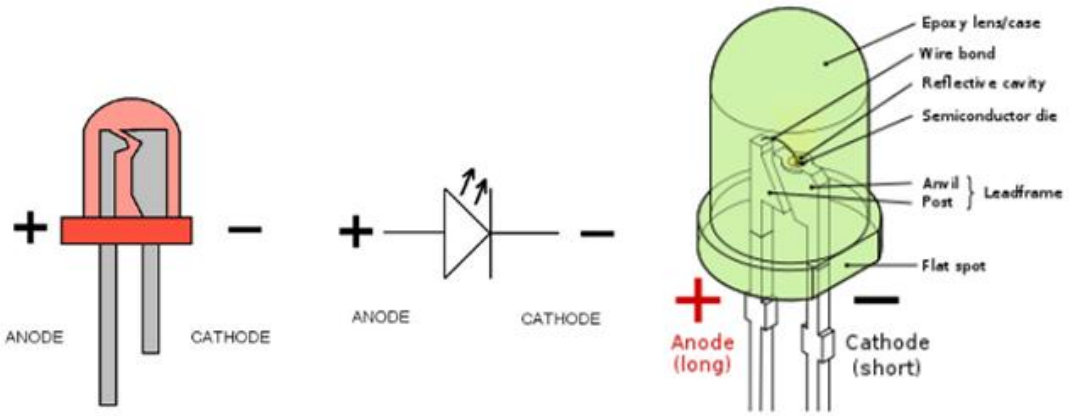
IR module: Infrared radiation is an electromagnetic wave with wavelength of 700nm to 1 mm. It is emitted by objects with temperature above 0 kelvins. Furthermore intensity and wavelength of infrared radiation depends on the temperature of the object. The infrared sensors are the sensors that detect/measure infrared radiation or change in the radiation from outer source or inbuilt source. Also sensors that use the property of infrared radiations to detect the changes in surrounding are termed as infrared sensors. An infrared sensor is an electronic instrument which is used to sense certain characteristics of its surroundings by either emitting and/or detecting infrared radiation. Infrared sensors are also capable of measuring the heat being emitted by an object and detecting motion.

LED: A light-emitting diode (LED) is a two-lead semiconductor light source. It is a p–n junction diode that emits light when activated When a suitable current is applied to the leads; electrons are able to recombine with electron holes within the device, releasing energy in the form of photons. This effect is called electroluminescence, and the color of the light (corresponding to the energy of the photon) is determined by the energy band gap of the semiconductor. LEDs are typically small (less than 1 mm2) and integrated optical components may be used to shape the radiation pattern.



A buzzer or beeper is an audio signalling device, which may be mechanical, electromechanical, or piezoelectric (piezo for short). Typical uses of buzzers and beepers include alarm devices, timers, and confirmation of user input such as a mouse click or keystroke. There are three types of buzzers: Electromechanical, Mechanical and Piezoelectric. Modern applications of buzzers include Novelty uses, judging panels, educational purposes, etc.

Draw block diagram /pin diagram of Raspberry-Pi/ Beagle board /Arduino Uno board interfacing with IR Sensor/Temperature Sensor/Camera. (Internal Examiner assign any one option for board and interface device and respective interface programming option) b. WAP in python/C++ language to blink LED. c. Write down the observations on Input and Output d. Write down the Result and Conclusion

Raspberry Pi GPIO Pin and interfacing:

A simple LED circuit consists of a LED and resistor. The resistor is used to limit the current that is being drawn and is called a current limiting resistor. Without the resistor the LED would run at too high of a voltage, resulting in too much current being drawn which in turn would instantly burn the LED, and likely also the GPIO port on the Raspberry Pi.

Pin#	NAME	Connection	Connection	NAME	Pin#
01	3.3V		5V (Cupcade)	5V	02
03	GPIO 2		5V (Powerboost)	5V	04
05	GPIO 3		GND (Powerboost)	Ground	06
07	GPIO 4	START		GPIO 14	08
09	Ground	GND (Cupcade)		GPIO 16	10
11	GPIO 17	UP	SELECT	GPIO 18	12
13	GPIO 27	DOWN	GND (Select/Start)	Ground	14
15	GPIO 22	LEFT	RIGHT	GPIO 23	16
17	3.3V		A	GPIO 24	18
19	GPIO 10	B	GND (ABXYR)	Ground	20
21	GPIO 09	X	Y	GPIO 25	22
23	GPIO 11	L Shoulder	R Shoulder	GPIO 08	24
25	Ground	GND (L)		GPIO 07	26
27	ID_SD			ID_SC	28
29	GPIO 05			Ground	30
31	GPIO 06			GPIO 12	32
33	GPIO 13			Ground	34
35	GPIO 19			GPIO 16	36
37	GPIO 26			GPIO 20	38
39	Ground			GPIO 21	40

With the circuit created we need to write the Python script to blink the LED. Before we start writing the software we first need to install the Raspberry Pi GPIO Python module. This is a library that allows us to access the GPIO port directly from Python. To install the Python library open a terminal and execute the following $ sudo apt-get install python-rpi.gpio python3-rpi.gpio With the library installed now open your favorite Python IDE (recommend Thonny Python IDE more information about using it here). Script needs to do the following: Initialize the GPIO ports Turn the LED on and off in 1 second intervals Procedure: Note: The Raspbian operating system comes with Python already installed in it. 1. Setting up the circuit: a. Turn OFF the Raspberry Pi while building/connecting the circuit 6 board/components according to diagram. b. Then turn the Raspberry Pi and check the Raspbian OS loaded properly or not. If not then check the circuit connection again.

Program1: Python program to blink LED

```
import RPi.GPIO as GPIO    # Import Raspberry Pi GPIO library

import time

# Import time module

GPIO.setwarnings(False)

# Ignore warning for now
```

```
GPIO.setmode(GPIO.BCM)

#assign numbering for the GPIO using BCM

#GPIO.setmode(GPIO.BOARD)

#assingn number for the GPIO using Board

cnt = 0

Blink_Time = 1

# change LED status every 1 seconds

RED_LED = 14

GPIO.setup(RED_LED, GPIO.OUT)

while True:

if cnt == 0 :

GPIO.output(RED_LED, False)

cnt = 1

else:

GPIO.output(RED_LED, True)

cnt = 0

time.sleep(Blink_Time)

GPIO.cleanup()
```

Program2: Python program to toggle two LED's.

```
import RPi.GPIO as GPIO  # Import Raspberry Pi GPIO library

from time import sleep

# Import the sleep function from the time module

GPIO.setwarnings(False)  # Ignore warning for now

GPIO.setmode(GPIO.BOARD) # Use physical pin numbering

RED_LED = 14

GREEN_LED = 15

GPIO.setup(RED_LED, GPIO.OUT, initial=GPIO.LOW)
```

```
GPIO.setup(GREEN_LED, GPIO.OUT, initial=GPIO.LOW)

while True:

# Run forever

GPIO.output(RED_LED, True) # Turn ON

GPIO.output(GREEN_LED, False) # Turn OFF

sleep(1) # Sleep for 1 second

GPIO.output(RED_LED, False) # Turn OFF

GPIO.output(GREEN_LED, True) # Turn ON

sleep(1) # Sleep for 1 second
```

Program3: Python program to turn ON/OFF buzzer.

```
import RPi.GPIO as GPIO  # Import Raspberry Pi GPIO library

from time import sleep

8

# Import the sleep function from the time module

GPIO.setwarnings(False)  # Ignore warning for now

GPIO.setmode(GPIO.BOARD) # Use physical pin numbering

Buzzer = 18

GPIO.setup(Buzzer, GPIO.OUT, initial=GPIO.LOW)

while True:

# Run forever

GPIO.output(Buzzer, True) # Turn ON

sleep(1) # Sleep for 1 second

GPIO.output(Buzzer, False) # Turn OFF

sleep(1) # Sleep for 1 second
```

Observations:

1. Input:

Any obstacle that comes in the range of the sensor is detected. IR sensors actually measure the heat being emitted from the object. So the heat is the actual input for the sensors. The IR sensor gets its input from GPIO 21.

2. Output:

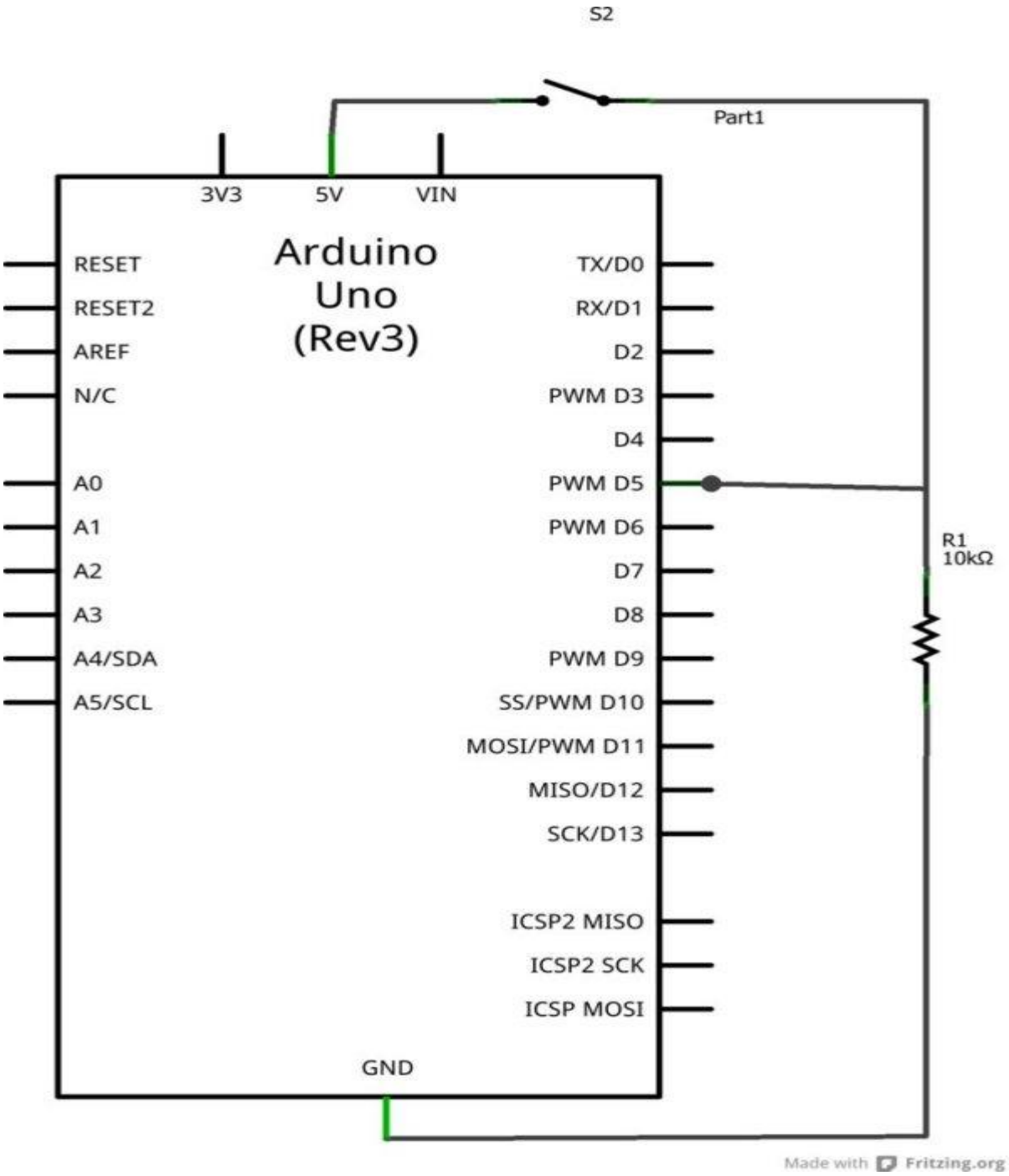
The output is shown by the LED's and the buzzer. When an obstacle is detected, the Green LED glows and buzzer is turned on, i.e. whenever heat is sensed by the sensor, output is shown.

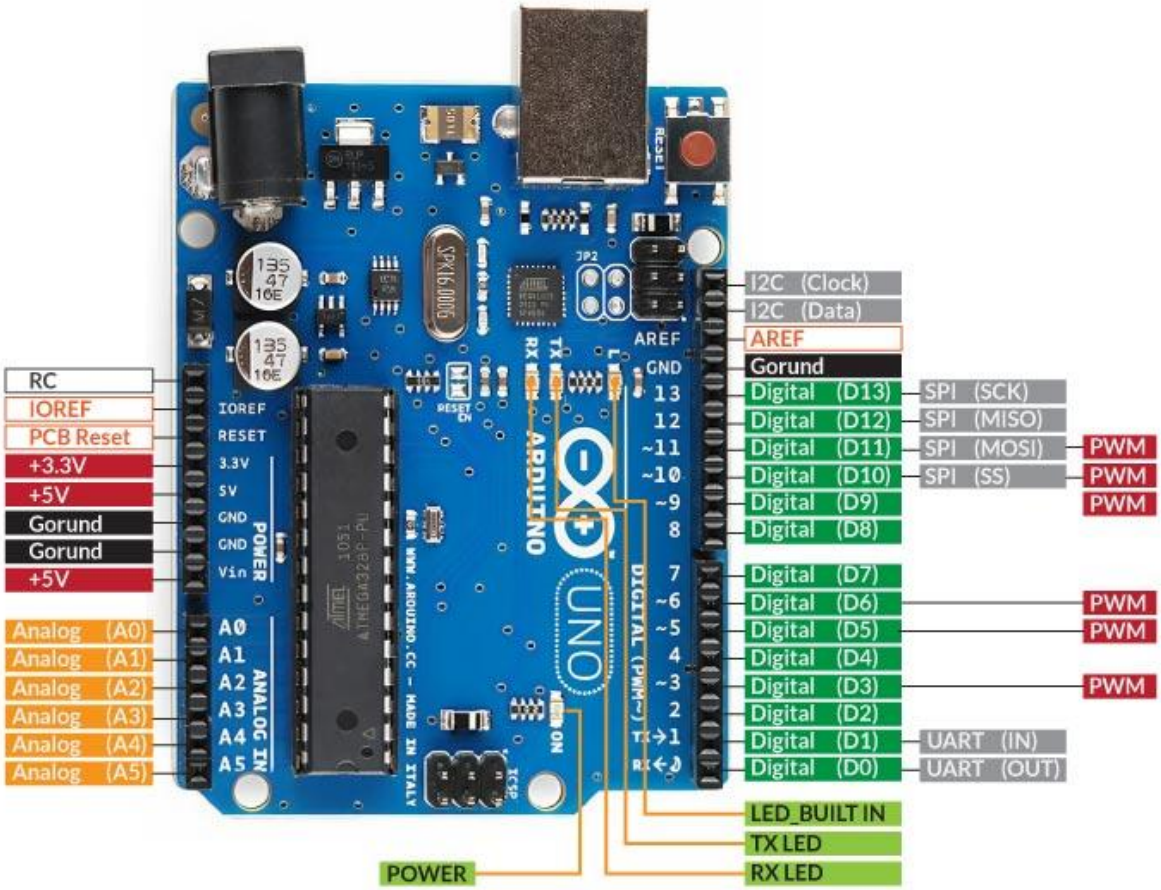
The output is shown by making the GPIO 15(Green LED) and GPIO 18 (buzzer) HIGH.

Conclusion:

We have successfully implemented the connection of IR sensor with Raspberry Pi/Beagle bone/Arduino for obstacle detection. The output is shown by a glowing LED's and buzzer.

Program For Practical




RC
IOREF
PCB Reset
+3.3V
+5V
Gorund
Gorund
+5V
Analog (A0)
Analog (A1)
Analog (A2)
Analog (A3)
Analog (A4)
Analog (A5)
POWER
ANALOG IN
i2C (Clock)
i2C (Data)
AREF
Gorund
Digital (D13)
Digital (D12)
Digital (D11)
Digital (D10)
Digital (D9)
Digital (D8)
Digital (D7)
Digital (D6)
Digital (D5)
Digital (D4)
Digital (D3)
Digital (D2)
Digital (D1)
Digital (D0)
SPI (SCK)
SPI (MISO)
SPI (MOSI)
SPI (SS)
PWM
PWM
PWM
PWM
PWM
PWM
UART (IN)
UART (OUT)
LED_BUILT IN
TX LED
RX LED
POWER
DIGITAL (PWM~)
UNO
ARDUINO

1.WAP in python/C++ language to blink LED.

```
void setup() {
// initialize digital pin LED_BUILTIN as an output.
pinMode(LED_BUILTIN, OUTPUT);
}
// the loop function runs over and over again forever
void loop() {
digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
delay(1000);
// wait for a second
```

```
digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
delay(1000);
// wait for a second
}
```

Observations:

1. Input: Any obstacle that comes in the range of the sensor is detected. IR sensors actually measure the heat being emitted from the object. So the heat is the actual input for the sensors. The IR sensor gets its input from GPIO 21.

2. Output: The output is shown by the LED's and the buzzer. When an obstacle is detected, the Green LED glows

Conclusion: We have successfully implemented the connection of IR sensor with Arduino for obstacle detection. The output is shown by a glowing buzzer.

2. WAP in python/C++ language to toggle two LED's.

```
/ Define the LED pins
const int led1 = 13;  // Built-in LED (usually on pin 13)
const int led2 = 2;   // External LED connected to pin 2
void setup() {
// Initialize both pins as output
pinMode(led1, OUTPUT);
pinMode(led2, OUTPUT);
}
void loop() {
// Turn on LED1 and turn off LED2
```

```
digitalWrite(led1, HIGH);
digitalWrite(led2, LOW);
delay(1000); // Wait for 1 second
// Turn off LED1 and turn on LED2
digitalWrite(led1, LOW);
digitalWrite(led2, HIGH);
delay(1000); // Wait for 1 second
}
```

Observations:

1. Input: Any obstacle that comes in the range of the sensor is detected. IR sensors actually measure the heat being emitted from the object. So the heat is the actual input for the sensors. The IR sensor gets its input from GPIO 21.

2. Output: The output is shown by the LED's and the buzzer. When an obstacle is detected, the Green LED glows

Conclusion: We have successfully implemented the connection of IR sensor with Arduino for obstacle detection. The output is shown by a glowing buzzer.

3. WAP in python/C++ language to turn ON/OFF buzzer.

```
// Define the buzzer pin
const int buzzerPin = 9;  // Connect the piezo buzzer to pin 9
// Define the frequency for the "tick" sound (in Hz)
const int tickFrequency = 1000;  // Frequency of the tick sound (1000 Hz)
// Define the duration for the "tick" sound
const int tickDuration =200;  // Duration of each tick sound (in milliseconds)
void setup() {
```

```
// Set the buzzer pin as an output

pinMode(buzzerPin, OUTPUT);

}
```